

# Programmentwurf Konsolen-Schach

**Name:** Max Katzenberger

**Matrikelnummer:** 4008573

**Name:** Mathis Koeder

**Matrikelnummer:** 3360189

**Abgabedatum:** 31.05.2026

# Kapitel 1: Einführung

## Übersicht über die Applikation

Die Anwendung ist ein vollständiges Konsolen-Schachspiel im Hot-Seat-Modus. Zwei Spieler teilen sich einen Rechner, geben ihre Züge nacheinander in algebraischer Notation ein (e2-e4, e7-e8=Q, 0-0) und sehen das Brett zwischen den Zügen als Unicode-ASCII-Ausgabe.

Implementiert sind alle regulären Schachregeln einschließlich Rochade, En passant, Bauernumwandlung sowie der Endspielerkennung Schachmatt, Patt, 50-Züge-Regel und dreifacher Stellungswiederholung. Partien können in einer einfachen, von Hand lesbaren Datei gespeichert und später wieder geladen werden (Format: FEN-Stellung plus PGN-light-Zugfolge).

Das Projekt löst kein konkretes Praxisproblem, sondern dient als didaktischer Übungsträger für Clean Architecture, Domain Driven Design, SOLID, Refactorings und Entwurfsmuster im Sinne des DHBW-Programmmentwurfs.

## Wie startet man die Applikation?

Voraussetzungen: JDK 17 oder neuer und ein Terminal mit Unicode-Unterstützung. Der mitgelieferte Gradle-Wrapper lädt sich Gradle 9 selbst herunter.

```
cd Chess
./gradlew run
```

Im Spiel führt der Befehl `help` eine Übersicht aller Eingaben auf. Eine typische Sitzung:

```
> new Max Mathis
Neue Partie: 4f3c2a-...
> e2-e4
> e7-e5
> save
> quit
```

## Wie testet man die Applikation?

Die JUnit-5-Suite läuft komplett über Gradle. Mockito-Tests benötigen das Flag `net.bytebuddy.experimental=true`, das im `build.gradle.kts` bereits gesetzt ist.

```
./gradlew test
./gradlew jacocoTestReport
open build/reports/jacoco/test/html/index.html
```

## Kapitel 2: Clean Architecture

### Was ist Clean Architecture?

Clean Architecture nach Robert C. Martin organisiert ein System in konzentrische Schichten, deren Abhängigkeiten ausschließlich nach innen gerichtet sind. Innerhalb dieser Schale liegen die Domänen-Entitäten und Geschäftsregeln, in den äußeren Schichten Anwendungs-Use-Cases und schließlich Adapter wie UI, Datenbank oder Frameworks. Die zentrale Regel *Dependency Rule* besagt, dass Code aus einer äußeren Schicht zwar auf innere Schichten zugreifen darf, aber niemals umgekehrt: die Domain weiß nichts von der Datenbank, die Datenbank weiß nichts vom UI.

Im Projekt ergeben sich vier Schichten:

- **domain:** Entitäten (`Game`, `Board`, `Piece...`), Value Objects, Domain-Services, Repository-Interfaces.
- **application:** Use-Cases als Application Services (`GameService`, `MoveService`, `PersistenceService`), Factories, DTOs.
- **infrastructure:** Implementierungen der Repository-Interfaces sowie Serialisierer (`FileGameRepository`, `FenSerializer`).
- **presentation:** Konsolen-UI mit `ConsoleApp`, `BoardRenderer`, `InputParser` und Command-Objekten.

Abhängigkeitsrichtung:  $\text{presentation} \rightarrow \text{application} \rightarrow \text{domain} \leftarrow \text{infrastructure}$ .

### Analyse der Dependency Rule

#### Positiv-Beispiel: Dependency Rule

`de.dhbw.chess.domain.entity.Game` ist Aggregat-Wurzel und liegt damit in der innersten Schicht. Die Klasse importiert ausschließlich aus `domain.valueobject`, `domain.entity` und `domain.service`, keinen Import in Richtung `application`, `infrastructure` oder `presentation`.

| Game                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - id : UUID<br>- white, black : Player<br>- board : Board<br>- history : MoveHistory<br>- validator : MoveValidator<br>- checkDetector : CheckDetector<br>- stateEvaluator : GameStateEvaluator<br>- activeColor : PieceColor<br>- status : GameStatus |
| + makeMove(move : Move) : MoveRecord<br>+ resign(color : PieceColor)                                                                                                                                                                                   |

**Wer hängt von Game ab?** `GameService` und `MoveService` aus der Application-Schicht, sowie alle Command-Klassen der Presentation-Schicht, also ausschließlich äußere Schichten. **Wovon hängt Game ab?** Nur von Klassen seiner eigenen Domain. Damit erfüllt die Klasse die Dependency Rule.

### Negativ-Beispiel: Dependency Rule

Im aktuellen Stand existiert keine Domain- oder Application-Klasse, die nach außen importiert. Als hypothetisches Negativ-Beispiel: würde `Game` direkt eine `FileGameRepository`-Instanz besitzen und beim `makeMove` synchron schreiben, würde der Domänenkern an die Infrastruktur-Schicht gekoppelt; ein Wechsel auf eine Datenbank zöge Änderungen mitten in der Domain nach sich. `MoveService` in Application als neuer Service. `game.makeMove(...)` bleibt in der Domain isoliert, der Service ruft anschließend `repository.save(game)` auf.

### Analyse der Schichten

#### Schicht: Domain — MoveValidator

`de.dhbw.chess.domain.service.MoveValidator` ist ein Domain-Service. Er prüft für eine Stellung, ob ein Zug regelkonform ist (Bewegungsmuster der Figur, Rochade-Vorbedingungen, En passant, Selbst-Schach). Es gibt keinen sinnvollen Identitätsbegriff, daher Service statt Entity. Die Klasse spricht nur Domain-Klassen an (`Board`, `Piece-Hierarchie`, `MoveHistory`) und ist damit Teil der innersten Schicht.

| <b>MoveValidator</b>                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------|
| - <code>checkDetector : CheckDetector</code><br>- <code>castlingRules : CastlingRules</code><br>- <code>enPassantRules : EnPassantRules</code> |
| + <code>validate(board, move, color, history)</code>                                                                                           |

#### Schicht: Infrastructure — FileGameRepository

`de.dhbw.chess.infrastructure.persistence.FileGameRepository` implementiert das Domain-Interface `GameRepository` und schreibt jede Partie als Textdatei ins Verzeichnis `~/.chess-saves/`. Die Klasse kennt die Domain (sie serialisiert `Board` und `MoveHistory`), die Domain kennt aber nicht die Datei.

| <<interface>><br><b>GameRepository</b>                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------|
| + <code>save(game : Game)</code><br>+ <code>findById(id : UUID)</code><br>: <code>Optional&lt;Game&gt;</code><br>+ <code>delete(id : UUID)</code> |
| <b>FileGameRepository</b>                                                                                                                         |
| - <code>directory : Path</code>                                                                                                                   |

## Kapitel 3: SOLID

### Analyse Single-Responsibility-Principle (SRP)

#### Positiv-Beispiel

`CheckDetector` hat genau eine Verantwortung: zu prüfen, ob der König einer Farbe auf einem gegebenen Brett im Schach steht.

| <b>CheckDetector</b>                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + <code>isInCheck(board : Board, color : PieceColor) : boolean</code><br>+ <code>isAttacked(board : Board, pos : Position, by : PieceColor) : boolean</code> |

Es gibt nur einen Grund, die Klasse zu ändern: wenn sich die Definition von „im Schach“ ändert. Sie kennt weder das Aggregat noch Persistenz und mischt keine Verantwortungen.

#### Negativ-Beispiel

In den Commits `39706e1` („`Game.makeMove` with inline validation“) und `a4d18f5` („inline check detection in `Game`“) hatte `Game` mehrere Aufgaben gleichzeitig: Aggregat-Lebenszyklus, Zugvalidierung *und* Schach-Erkennung. Die gesamte Regel-Logik hing direkt in `makeMove`, sodass die Klasse mehr als einen Grund hatte, sich zu ändern — jedes neue Regeldetail (Rochade, En passant, Selbst-Schach) hätte mitten in `Game` eingeflochten werden müssen.

Listing 1: `Game.makeMove` (Stand `39706e1`) — Validierung inline im Aggregat

```
public MoveRecord makeMove(Move move) {
    Objects.requireNonNull(move, "move");
    if (status.isFinal()) {
        throw new IllegalStateException("Partie ist beendet");
    }
    Piece moving = board.pieceAt(move.from());
    if (moving == null) {
        throw new IllegalArgumentException("Kein Stueck auf " +
            move.from());
    }
    if (moving.color() != activeColor) {
        throw new IllegalArgumentException("Falsche Farbe am Zug");
    }
    List<Position> targets = moving.possibleMoves(board, move.from());
    if (!targets.contains(move.to())) {
        throw new IllegalArgumentException("Zug nicht erlaubt fuer " + moving.type());
    }
    // ... Brett aendern, Historie schreiben, Farbe wechseln
}
```

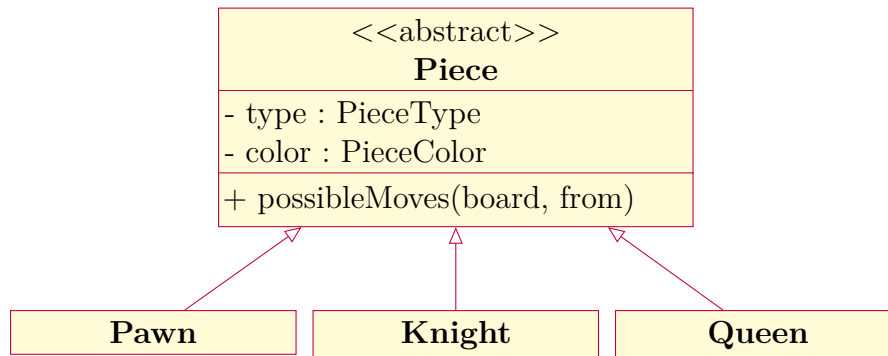
}

Lösungsweg: `MoveValidator` und `CheckDetector` als eigene Domain-Services extrahieren. `Game` delegiert dann nur noch und bleibt damit auf seine Aggregat-Verantwortung beschränkt.

## Analyse Open-Closed-Principle (OCP)

### Positiv-Beispiel

Die abstrakte Klasse `Piece` mit der Methode `possibleMoves(Board, Position)` und den Subklassen `Pawn`, `Rook`, `Bishop`, `Queen`, `Knight`, `King`.



Soll eine neue Figur (z. B. „Berolina-Bauer“) hinzukommen, schreibt man eine neue Subklasse, ohne bestehende Klassen zu ändern — offen für Erweiterung, geschlossen für Modifikation. Das gleiche Prinzip nutzt das Command-Pattern in der Presentation-Schicht: ein neuer CLI-Befehl ist eine neue `Command`-Implementation; der Dispatcher in `ConsoleApp` bleibt unangetastet.

### Negativ-Beispiel

Im Commit 38655b6 („Piece base class with type-based switch logic“) existierte eine einzelne, finale `Piece`-Klasse mit einer langen `switch(type)`-Anweisung in `possibleMoves`. Jede neue Figur hätte den `switch` aufgebrochen. Der Code-Smell wurde im Refactoring 1 (Commit 4a5b875) durch Extraktion in Subklassen (Strategy) aufgelöst.

Listing 2: `Piece.possibleMoves` (Stand 89e3843) — zentrale switch-Logik

```
public List<Position> possibleMoves(Board board, Position from) {
    List<Position> moves = new ArrayList<>();
    switch (type) {
        case PAWN:
            addPawnMoves(board, from, moves);
            break;
        case ROOK:
            addSlidingMoves(board, from, moves,
                new int [][] {{1,0},{-1,0},{0,1},{0,-1}});
            break;
        case BISHOP:
            addSlidingMoves(board, from, moves,
                new int [][] {{1,1},{1,-1},{-1,1},{-1,-1}});
            break;
        case QUEEN:
            addSlidingMoves(board, from, moves, new int [][] {
                {1,0},{-1,0},{0,1},{0,-1},
                {1,1},{1,-1},{-1,1},{-1,-1}});
            break;
        case KNIGHT:
            addStepMoves(board, from, moves, new int [][] {
```

```

        {1,2},{2,1},{-1,2},{-2,1},
        {1,-2},{2,-1},{-1,-2},{-2,-1}});
    break;
case KING:
    addStepMoves(board, from, moves, new int[][] {
        {1,0},{-1,0},{0,1},{0,-1},
        {1,1},{1,-1},{-1,1},{-1,-1}});
    break;
default:
    throw new IllegalStateException("Unbekannter Figurentyp: " +
        type);
}
return moves;
}

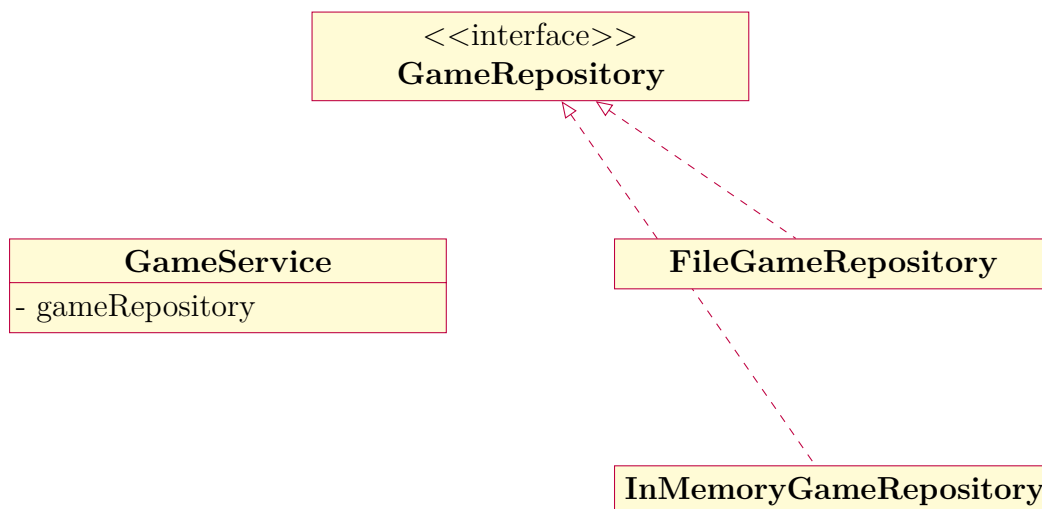
```

## Analyse Liskov-Substitution- (LSP), Interface-Segregation- (ISP), Dependency-Inversion-Principle (DIP)

Hier wird das **Dependency-Inversion-Principle (DIP)** ausgewählt.

### Positiv-Beispiel

GameService hängt vom *Interface* GameRepository ab, nicht von einer konkreten Implementierung. Im Konstruktor wird das Repository injiziert; ob dahinter InMemoryGameRepository, FileGameRepository oder im Test ein Mockito.mock(GameRepository.class) steht, ist transparent.



Die Application-Schicht hängt nur in Richtung Abstraktion (Interface in der Domain), die Infrastruktur-Schicht implementiert dieselbe Abstraktion — klassisches DIP.

### Negativ-Beispiel

Würde ConsoleApp direkt `new FileGameRepository(...)` bauen *und* dann selbst Dateipfade konstruieren und FEN parsen, hinge die Presentation-Schicht an einer konkreten Klasse statt an einer Abstraktion. Aktuell instanziiert ConsoleApp zwar das konkrete



`FileGameRepository` (Zusammenstellung in `main`), reicht es aber als `GameRepository`-Referenz an die Services weiter, sodass die übrige Anwendung weiterhin gegen die Abstraktion programmiert ist.

## Kapitel 4: Weitere Prinzipien

### Analyse GRASP: Geringe Kopplung

#### Positiv-Beispiel

`GameService` kennt nur das `GameRepository`-Interface und die `GameFactory`. Wechselt die Persistenzstrategie, ändert sich am Service nichts. Insgesamt verläuft Kommunikation in diesem Projekt fast ausschließlich über schmale Interfaces und Value Objects.

#### Negativ-Beispiel

Eine engere Kopplung entsteht in der Presentation-Schicht beim CLI-Befehl `MoveCmd`: das Command-Objekt kennt nicht nur den `MoveService`, sondern greift zusätzlich direkt auf das Domain-Modell zu (`game.board()`, `game.activeColor()`, `King.class`), um bei einer Rochade-Eingabe (0-0) das Königsfeld zu finden. Damit hängt eine Presentation-Klasse an mehreren Domain-Klassen gleichzeitig (`Game`, `Board`, `King`, `Position`). Die Kopplung ist gewollt, aber nicht minimal — denkbar wäre, einen Application-Service-Aufruf wie `moveService.executeCastling(gameId, kingside)` einzuführen, sodass die CLI keinen Zugriff mehr auf Domain-Entities benötigt.

Listing 3: `MoveCmd` — Presentation-Klasse mit direktem Zugriff auf Domain-Entities

```
private Position findKing(Game game, PieceColor color) {
    for (int f = 0; f < 8; f++) {
        for (int r = 0; r < 8; r++) {
            Position p = Position.of(f, r);
            if (game.board().pieceAt(p) instanceof King k && k.color()
                == color) {
                return p;
            }
        }
    }
    throw new IllegalStateException("Kein Koenig auf dem Brett");
}
```

### Analyse GRASP: Hohe Kohäsion

`FenSerializer` hat ausschließlich eine Aufgabe: das Brett-Layout einer Schachpartie zwischen Java-Objekten und der Standard-FEN-Stellungsnotation zu konvertieren. Beide Methoden (`serializePlacement` und `parsePlacement`) gehören eng zusammen, teilen den Wertebereich (FEN-Symbole) und werden im selben Use-Case — Speichern und Laden — benötigt. Entsprechend hoch ist die Kohäsion.

### Don't Repeat Yourself (DRY)

Im früheren Stand (Commit `a4d18f5`) enthielt `Game` sowohl die Validierung eines Zugs als auch eine eigenständige Schach-Erkennung. Beide Stellen prüften, ob ein Probe-Brett den eigenen König in Schach setzen würde — die Logik (Brett kopieren, Zug ausführen, Königsfeld auf Angriffsfreiheit prüfen) war doppelt vorhanden.

Listing 4: Game (Stand a4d18f5) — Schach-Prüfung inline und dupliziert

```
public MoveRecord makeMove(Move move) {
    // ... (Eigentums- und Zielfeld-Pruefung) ...
    board.move(move.from(), move.to());
    boolean isCheck = isInCheck(activeColor.opposite());
    if (isInCheck(activeColor)) {
        board.move(move.to(), move.from());
        throw new IllegalArgumentException("Zug laesst eigenen Koenig im
            Schach");
    }
    // ...
}

private boolean isInCheck(PieceColor color) {
    Position kingPos = board.findKing(color);
    if (kingPos == null) return false;
    for (int f = 0; f < Board.SIZE; f++) {
        for (int r = 0; r < Board.SIZE; r++) {
            Position from = Position.of(f, r);
            Piece p = board.pieceAt(from);
            if (p == null || p.color() == color) continue;
            if (p.possibleMoves(board, from).contains(kingPos)) {
                return true;
            }
        }
    }
    return false;
}
```

Im Refactoring 2 (Commits 82a8048, cede2d0) wurden `MoveValidator` und `CheckDetector` extrahiert; seitdem existiert die Schach-Prüfung an genau einer Stelle und beide Aufrufer (Validator beim Probespielen, Aggregat beim Erkennen von „Schach!“) rufen denselben Service auf.

Listing 5: CheckDetector (Stand cede2d0) — Schach-Prüfung an genau einer Stelle

```
public class CheckDetector {

    public boolean isInCheck(Board board, PieceColor color) {
        Position kingPos = board.findKing(color);
        if (kingPos == null) return false;
        return isAttacked(board, kingPos, color.opposite());
    }

    public boolean isAttacked(Board board, Position target, PieceColor
        attacker) {
        for (int f = 0; f < Board.SIZE; f++) {
            for (int r = 0; r < Board.SIZE; r++) {
                Position from = Position.of(f, r);
                Piece p = board.pieceAt(from);
                if (p == null || p.color() != attacker) continue;
                if (p.possibleMoves(board, from).contains(target)) {
                    return true;
                }
            }
        }
    }
}
```

```
    }  
    return false;  
}  
}
```

## Kapitel 5: Unit Tests

### 10 Unit Tests

| Unit Test | Beschreibung                                           | Klasse#Methode                                        |
|-----------|--------------------------------------------------------|-------------------------------------------------------|
| UT01      | Position weist negative oder zu große Indizes ab.      | PositionTest#rejectsOutOfBounds                       |
| UT02      | PieceColor.opposite() ist symmetrisch.                 | PieceColorTest#oppositeIsSymmetric                    |
| UT03      | Move.equals/hashCode ist konsistent.                   | MoveTest#equalsAndHashCode                            |
| UT04      | Springer-Sprung-Muster sind korrekt.                   | PieceMovesTest#knightLShape                           |
| UT05      | Bauern-Doppelschritt nur aus Grundreihe.               | PieceMovesTest#pawnDoubleStep                         |
| UT06      | Kurze Rochade gelingt unter Standardbedingungen.       | CastlingTest#kingsideCastling                         |
| UT07      | En passant schlägt den passenden Bauern.               | EnPassantTest#capturesAdjacentPawnAfterDoubleStep     |
| UT08      | Promotion ersetzt den Bauern durch die gewählte Figur. | PromotionTest#promotionToQueenReplacesPawn            |
| UT09      | Schachmatt setzt GameStatus korrekt.                   | EndgameTest#backRankMateEndsGameWithWhiteWin          |
| UT10      | GameService delegiert Persistenz an das Repository.    | GameServiceMockTest#startNewGamePersistsViaRepository |

Insgesamt umfasst die Suite mehr als 80 Tests; die Tabelle zeigt einen repräsentativen Auszug.

### ATRIP: Automatic

Die Tests laufen ausschließlich über `./gradlew test`, ohne manuelle Eingaben oder externe Ressourcen. Dateibasierte Tests (`FileGameRepositoryRoundTripTest`) verwenden `@TempDir`, sodass kein gemeinsam genutztes Dateisystem nötig ist. Das Erfolgs-/Fehlensignal ist der Exit-Code des Gradle-Aufrufs — automatisierbar in jeder CI.

### ATRIP: Thorough

**Positiv-Beispiel:** `PromotionTest` prüft alle vier Promotion-Optionen (Queen, Rook, Bishop, Knight) sowie zwei Fehlerfälle: Promotion-Pflicht beim Erreichen der letzten Reihe und Ablehnung von Promotion außerhalb dieser Reihe. Damit deckt der Test den positiven wie den negativen Pfad gleichmäßig ab.

**Negativ-Beispiel:** `InputParserTest#rejectsMalformedInput` prüft drei Fehlerformen in einer Methode (`e2e4`, leerer String, null an `Position.fromAlgebraic`). Sauberer

wäre eine Aufteilung in dedizierte Tests pro Eingabeklasse, weil der erste Assert beim Fehlschlagen die übrigen verdeckt.

## ATRIP: Professional

**Positiv-Beispiel:** `EndgameTest#threefoldRepetitionEndsAsDraw` verwendet eine Zyklus-Schleife mit Statusabfrage, statt eine konkrete Halbzug-Anzahl hart zu kodieren. Damit bleibt der Test stabil, wenn sich die Implementierung der Wiederholung minimal ändert (etwa: ab welchem Halbzug der Zähler greift).

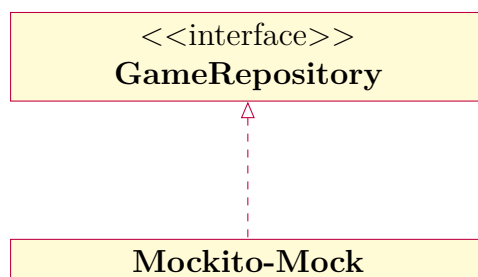
**Negativ-Beispiel:** Die Hilfsmethode `p(String)` ist in mehreren Test-Klassen copy-pasted (`CastlingTest`, `EnPassantTest`, `PromotionTest`, `EndgameTest`). Professioneller wäre eine gemeinsame Test-Utility, etwa `static import` aus einer `Squares`-Hilfsklasse.

## Code Coverage

`./gradlew jacocoTestReport` erzeugt einen HTML-Bericht unter `build/reports/jacoco/test/html`. Die Domain-Schicht ist mit den vorhandenen Tests gut abgedeckt: alle Figuren-Subklassen, `MoveValidator`, `CheckDetector`, `GameStateEvaluator` sowie `Game.makeMove` besitzen umfangreiche Pfad-Abdeckung. Application-Services werden über `GameServiceMockTest` und `PersistenceServiceTest` mit Mockito getestet. Die Presentation-Schicht ist bewusst nur partiell abgedeckt (`InputParserTest`); reine Glue-Code-Klassen wie `ConsoleApp` oder die einzelnen Command-Objekte werden manuell beim Spielstart verifiziert.

## Fakes und Mocks

**Mock 1: GameRepository.** In `GameServiceMockTest` und `PersistenceServiceTest` wird das Repository durch `Mockito.mock(...)` ersetzt. Damit können wir prüfen, dass der Service `save(...)` aufruft (`verify`) und das `findById`-Verhalten passend stubben (`when(repo.findById(id)).thenReturn(...)`), ohne tatsächlich Dateien zu schreiben.



**Mock 2: GameRepository im PersistenceService-Test.** Auch `PersistenceService` wird gegen einen Mock getestet; das Round-Trip-Szenario findet zusätzlich in einem realen `FileGameRepository` unter `@TempDir` statt, sodass Mock und Fake-Verhalten getrennt voneinander geprüft werden.

## Kapitel 6: Domain Driven Design

### Ubiquitous Language

| Bezeichnung | Bedeutung                                                                | Begründung                                                                                                   |
|-------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Game        | Eine vollständige Schachpartie mit Spielern, Brett, Zughistorie, Status. | Der zentrale Begriff aus der Schach-Domäne; sowohl Regelwerk als auch Spieler reden von „der Partie“.        |
| Position    | Algebraisches Feld auf dem Brett (a1–h8).                                | Schachspieler verwenden diese Notation seit Jahrhunderten. Code und UI sprechen denselben Begriff.           |
| Move        | Ein einzelner Halbzug (Quelle, Ziel, ggf. Promotion).                    | Die Domänensprache unterscheidet Halbzug von Vollzug; <b>Move</b> entspricht dem englischen Halbzug-Begriff. |
| Check       | Stellung, in der der König eines Spielers angegriffen ist.               | Schach / Check ist ein Kernkonzept der Regeln; im Code modelliert durch <b>CheckDetector</b> .               |

### Entities

**Game** ist eine Entity — die Identität wird durch eine **UUID** stabilisiert, der Zustand (Brett, Status, ziehende Farbe) ändert sich über die Zeit. Zwei Partien mit identischem Brett sind nicht dieselbe Partie.

| Game                                                                               |
|------------------------------------------------------------------------------------|
| - id : UUID<br>- board : Board<br>- history : MoveHistory<br>- status : GameStatus |
| + makeMove(move)<br>+ resign(color)                                                |

### Value Objects

**Position** ist ein klassisches Value Object: unveränderlich, definiert durch seine Werte (**file**, **rank**), Gleichheit über alle Felder. Zwei **Position.of(4, 1)** sind identisch, eine **Position** kennt keine Identität.

| Position                                             |
|------------------------------------------------------|
| - file : int<br>- rank : int                         |
| + toAlgebraic() : String<br>+ fileDelta(other) : int |

## Repositories

**GameRepository** ist die abstrakte Schnittstelle, über die Application-Services Partien laden und speichern, ohne Persistenz-Details zu kennen. Implementierungen liegen in der Infrastruktur-Schicht.

|                                                                 |
|-----------------------------------------------------------------|
| <b>&lt;&lt;interface&gt;&gt;</b><br><b>GameRepository</b>       |
| + save(game)<br>+ findById(id) : Optional<Game><br>+ delete(id) |

## Aggregates

**Game** ist die Aggregat-Wurzel. Es schließt **Board**, **MoveHistory**, **Player**, **GameStatus** und die ziehende Farbe als Konsistenzgrenze ein. Alle regelrelevanten Mutationen (**makeMove**, **resign**) laufen über **Game**, sodass Invarianten wie „Status-Wechsel nur nach gültigem Zug“ oder „aktive Farbe wechselt nach jedem Zug“ an einer Stelle gewahrt bleiben. Externe Klassen erhalten das Brett nur als Read-Only-Sicht (Snapshot, Renderer).



## Kapitel 7: Refactoring

### Code Smells

**Code Smell 1 — Switch Statement (Commit 38655b6):** ursprünglich enthielt `Piece` ein `switch(type)` mit Zugregeln aller Figuren in einer Methode. Lösung war „Replace Conditional with Polymorphism“: Subklassen pro Figurentyp.

Listing 6: Code Smell 1 — switch in Piece (Stand 89e3843)

```
public List<Position> possibleMoves(Board board, Position from) {
    List<Position> moves = new ArrayList<>();
    switch (type) {
        case PAWN:    addPawnMoves(board, from, moves); break;
        case ROOK:    addSlidingMoves(board, from, moves, /* ... */);
                     break;
        case BISHOP:  addSlidingMoves(board, from, moves, /* ... */);
                     break;
        case QUEEN:   addSlidingMoves(board, from, moves, /* ... */);
                     break;
        case KNIGHT: addStepMoves(board, from, moves, /* ... */);
                     break;
        case KING:   addStepMoves(board, from, moves, /* ... */);
                     break;
        default: throw new IllegalStateException("Unbekannter Figurentyp
            : " + type);
    }
    return moves;
}
```

**Code Smell 2 — Large Class / Duplicated Logic (Commit a4d18f5):** im früheren Stand vereinte `Game` die Aggregat-Verantwortung mit Validierung und Schach-Erkennung; die Schach-Erkennung war zudem an zwei Stellen dupliziert. Lösung war „Extract Class“ zu `MoveValidator` und `CheckDetector`.

Listing 7: Code Smell 2 — `Game.makeMove` validiert und prüft Schach inline (Stand a4d18f5)

```
public MoveRecord makeMove(Move move) {
    // ... pseudo-legale Pruefung ...
    board.move(move.from(), move.to());
    boolean isCheck = isInCheck(activeColor.opposite()); // 1. Aufruf
    if (isInCheck(activeColor)) {                          // 2. Aufruf,
        gleiche Logik
        board.move(move.to(), move.from());
        throw new IllegalArgumentException("Zug laesst eigenen Koenig im
            Schach");
    }
    // ...
}
```

### 2 Refactorings

**Refactoring 1: Replace Conditional with Polymorphism (Commits 0bf79b1, 4a5b875).**

**Vorher:** eine Piece-Klasse mit `switch(type)` (siehe Code Smell 1 oben).

| Piece (vorher)                             |
|--------------------------------------------|
| - type : PieceType<br>- color : PieceColor |
| + possibleMoves(...) : List<Position>      |

**Nachher:** abstrakte Piece-Basis mit konkreten Subklassen Pawn, Rook, Bishop, Queen, Knight, King (sowie SlidingPiece als Zwischen-Schicht für Linien-Figuren). Jede Klasse trägt nur ihre eigene Zuglogik, das System ist offen für neue Figuren ohne Änderung am Bestand (OCP).

Listing 8: Refactoring 1 nachher (Stand 4a5b875) — Pawn als eigene Strategy-Klasse

```
public abstract class Piece {
    public abstract List<Position> possibleMoves(Board board, Position
        from);
    // ... gemeinsamer Zustand: type, color, hasMoved ...
}

public final class Pawn extends Piece {
    public Pawn(PieceColor color) { super(PieceType.PAWN, color); }

    @Override
    public List<Position> possibleMoves(Board board, Position from) {
        List<Position> moves = new ArrayList<>();
        int dir = color().isWhite() ? 1 : -1;
        // ... bauern-spezifische Zuglogik ...
        return moves;
    }
}
```

**Refactoring 2: Extract Class — MoveValidator und CheckDetector** (Commits 82a8048, cede2d0).

**Vorher:** Game mit Inline-Validierung und doppelter Schach-Prüfung (siehe Code Smell 2 oben).

| Game (vorher)                                                        |
|----------------------------------------------------------------------|
| + makeMove(move)<br>- validateInline(move)<br>- isInCheckInline(...) |

**Nachher:** Game delegiert die Prüfung an einen MoveValidator, dieser nutzt einen CheckDetector für die Selbst-Schach-Probe. Die Schach-Prüfung existiert nur noch einmal (DRY), und das Aggregat orchestriert.

| Game (nachher)                                                 | MoveValidator                   |
|----------------------------------------------------------------|---------------------------------|
| - validator : MoveValidator<br>- checkDetector : CheckDetector | - checkDetector : CheckDetector |

Listing 9: Refactoring 2 nachher (Stand 82a8048) — MoveValidator als eigener Domain Service

```
public class MoveValidator {

    private final CheckDetector checkDetector;

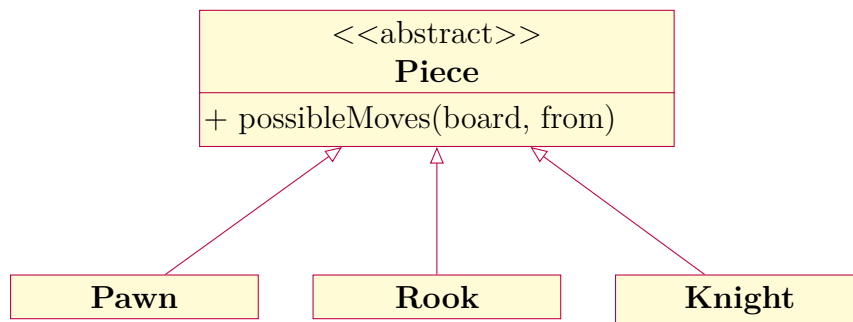
    public MoveValidator(CheckDetector checkDetector) {
        this.checkDetector = checkDetector;
    }

    public void validate(Board board, Move move, PieceColor activeColor)
    {
        Piece moving = board.pieceAt(move.from());
        if (moving == null)
            throw new IllegalArgumentException("Kein Stueck auf " + move
                .from());
        if (moving.color() != activeColor)
            throw new IllegalArgumentException("Falsche Farbe am Zug");
        if (!moving.possibleMoves(board, move.from()).contains(move.to()
        ))
            throw new IllegalArgumentException("Zug nicht erlaubt fuer "
                + moving.type());
        Board probe = board.copy();
        probe.move(move.from(), move.to());
        if (checkDetector.isInCheck(probe, activeColor))
            throw new IllegalArgumentException("Zug laesst eigenen
                Koenig im Schach");
    }
}
```

## Kapitel 8: Entwurfsmuster

### Entwurfsmuster: Strategy

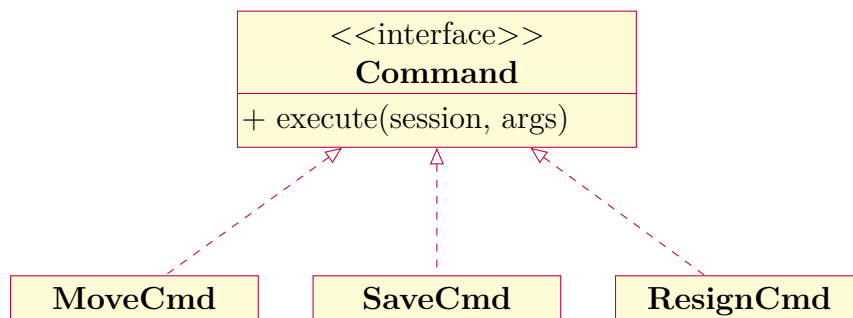
Die abstrakte Klasse `Piece` ist Strategie-Träger; jede konkrete Figurklasse (`Pawn`, `Rook`, `Bishop`, `Queen`, `Knight`, `King`) implementiert eine eigene Variante der `possibleMoves`-Strategie. Aus Sicht von `MoveValidator` und `Game` ist die Figur ein `Piece`, die Auswahl der korrekten Zuglogik geschieht polymorph.



Vorteil: das System ist offen für neue Figuren, der Validator und das Aggregat müssen für eine neue Figur nicht angefasst werden.

### Entwurfsmuster: Command

Alle CLI-Eingaben sind als Kommando-Objekte modelliert. Das Interface `Command` hat eine einzige Methode, der Dispatcher in `ConsoleApp` wählt anhand des Eingabe-Schlüsselworts das passende Objekt aus einer `Map`. Ein neuer Befehl ist eine neue Klasse plus ein `put`; bestehende Klassen bleiben unberührt (OCP).



Erweiterungsmöglichkeit: ein zukünftiges Undo lässt sich umsetzen, indem jedes Command beim Ausführen seinen Inversions-Zustand auf einem Stack ablegt — die übrige Anwendung muss dafür nicht umstrukturiert werden.